**Laboratorio #9:**

# Laboratorio 1:

**Objetivos:**

- Crear un pipeline CI/CD en GitLab para aplicaciones Python

- Crear una API REST con FastAPI

- Adaptar una app FastAPI para correr en AWS Lambda con Mangum

- Escribir un handler Lambda compatible con API Gateway v2

- Configurar Serverless Framework v3 (sin cuenta) con `serverless.yml`

- Levantar la API en local con `serverless offline` (API Gateway + Lambda simulados) y validarla desde el pipeline

- Instalar y registrar un GitLab Runner en tu laptop y enrutar jobs con `tags:`

- Distinguir en el log si un job corrió en tu laptop o en un shared runner

- Usar cache de pipeline (pip y npm) para acelerar ejecuciones

- Pasar artifacts entre jobs

- Entender el orden por stages y cuándo usar `needs:`

Nota: el despliegue a AWS Lambda (`serverless deploy`), la migración a Serverless Framework v4 y las credenciales (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `SERVERLESS_ACCESS_KEY`) se configuran en el laboratorio de Bitbucket Pipelines.

**Guías de laboratorio:**

- https://github.com/utec-devops-2026/laboratorios/blob/main/laboratorio_gitlab/guia_gitlab_ci_runner.md

## Entregables

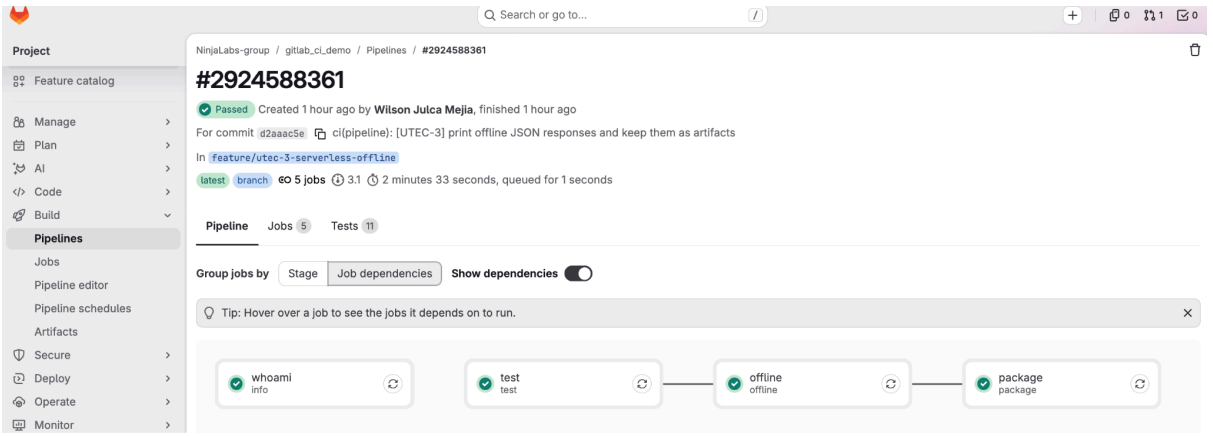
1. **URL de repositorio** (GitLab).
    - https://gitlab.com/<usuario>/gitlab_lambda_demo
2. Subir su repositorio a gitlab. Brindar acceso a usuario (@pytuxi)

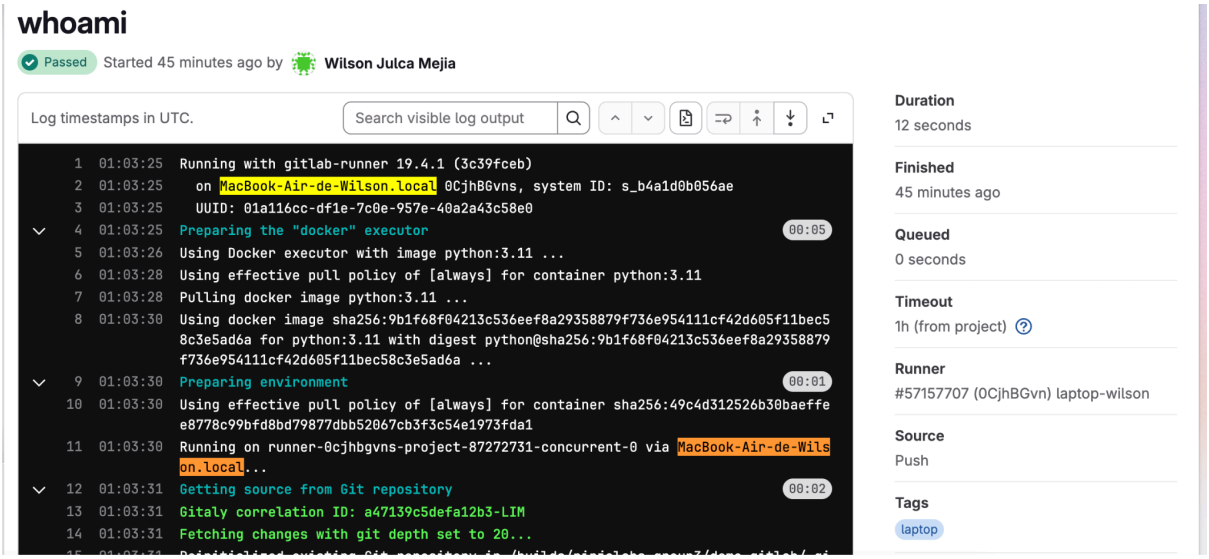
Estructura del repo:

```
gitlab_lambda_demo/
├── .gitlab-ci.yml            # Pipeline principal de CI/CD
├── handler.py                # Handler para AWS Lambda
├── hello.py                  # Aplicación Python (ejecución local)
├── serverless.yml            # Configuración de Serverless Framework
├── package.json              # Dependencias Node.js (serverless plugins)
├── tests/
│   ├── test_hello.py         # Tests unitarios de la API
│   └── test_handler.py       # Tests del handler Lambda (evento API Gateway v2)
├── requirements.txt          # Dependencias Python (producción / Lambda)
├── requirements-dev.txt      # Dependencias de desarrollo (lint, test)
├── .gitignore                # node_modules/, .serverless/, .venv/
└── README.md
```

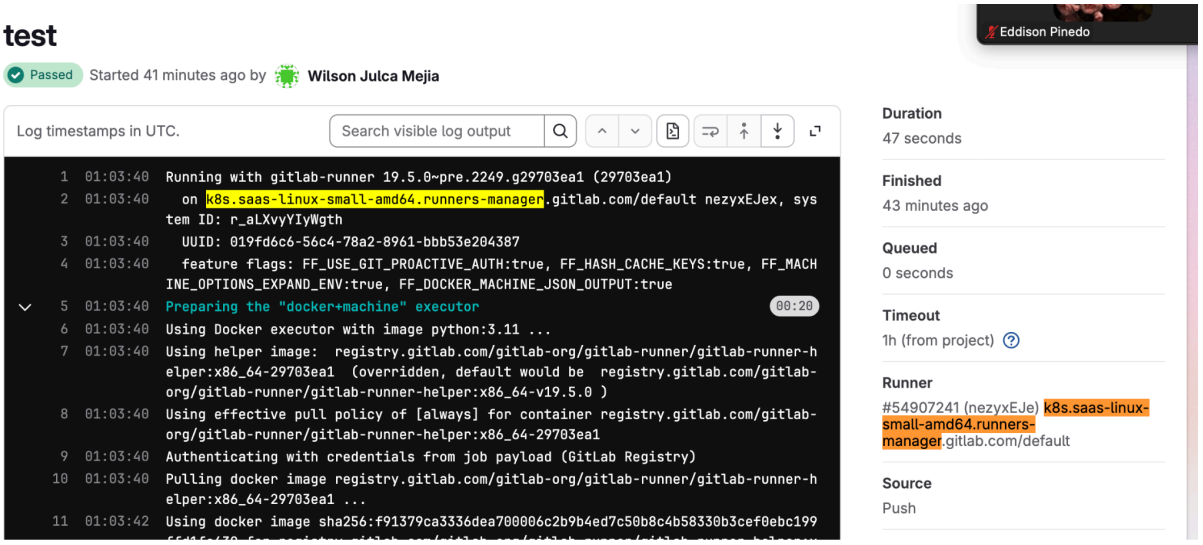
3. Captura del pipeline en verde con los jobs whoami, test, offline, package y docs

4. Captura de Settings → CI/CD → Runners con tu runner Online y del log del job whoami corriendo en tu laptop

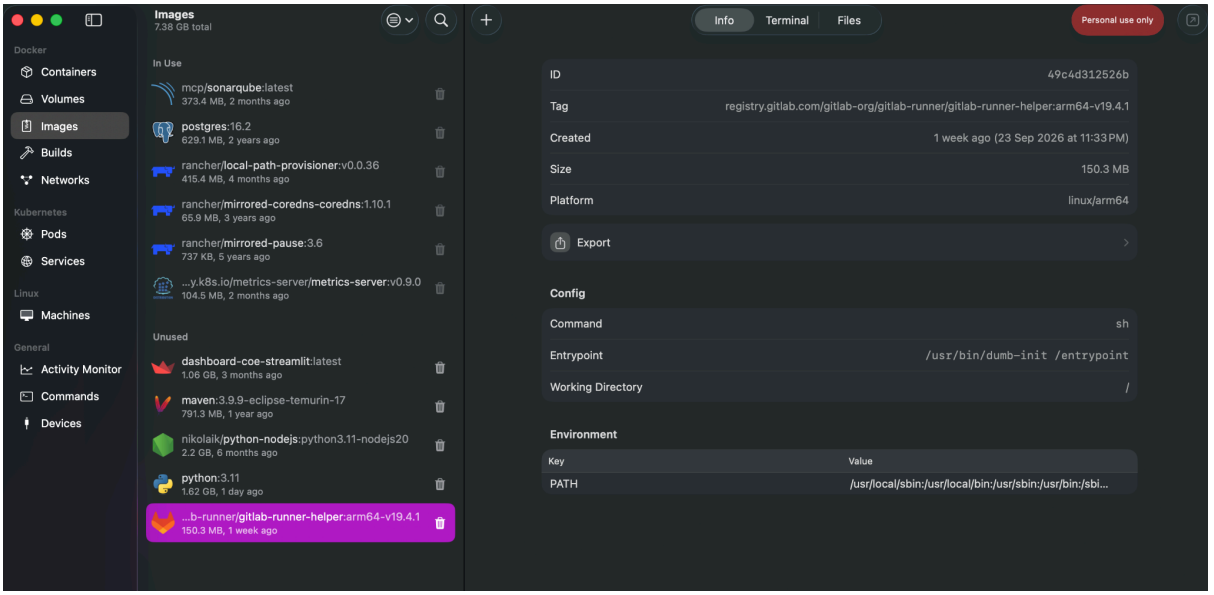





5. Captura de Settings → CI/CD → Runners con tu runner Online y del log del job test para validar que corre en la máquina de Gitlab




6. Captura de la imagen del runner de gitlab en docker desktop




7. Captura de la API respondiendo en local con serverless offline: terminal con `npx serverless offline` (Server ready) y curl o navegador en http://localhost:3000/hello?name=<tu-nombre>




8. Captura del log del job offline (Build → Jobs → offline) mostrando las líneas RESPUESTA /hello y RESPUESTA /add con el JSON. El artifact offline.log es el log del servidor serverless-offline (invocaciones λ), no contiene las respuestas.

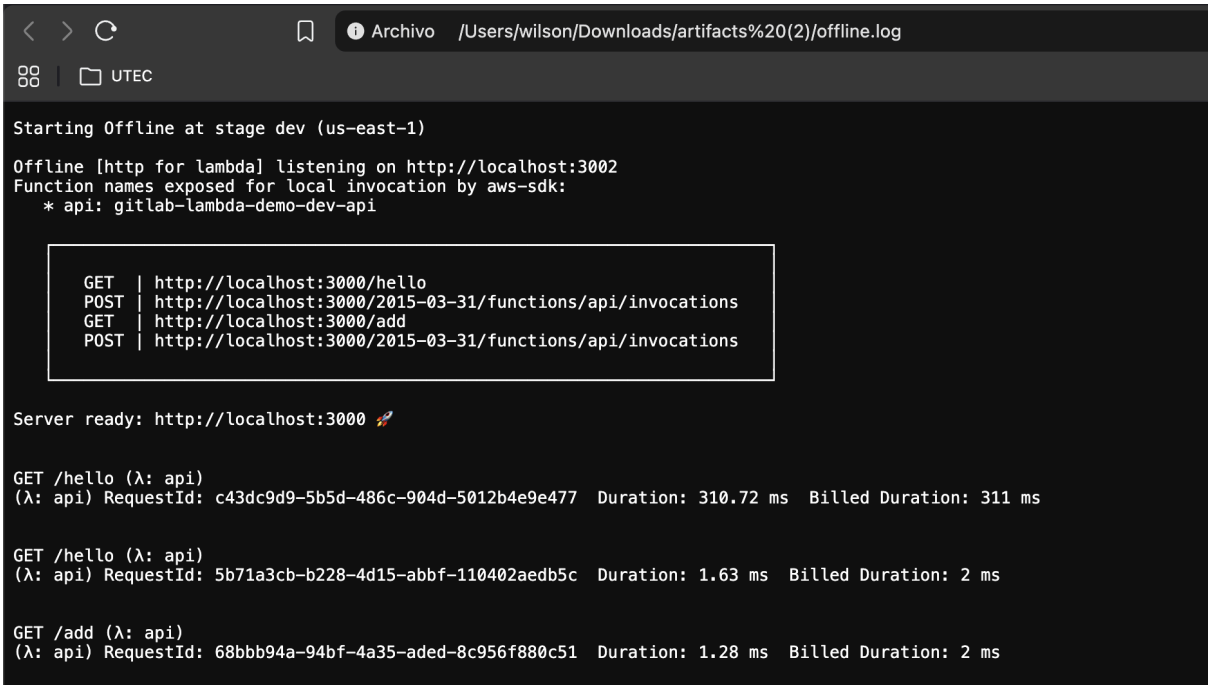
log: hello




log: add




log: offline


Archivo /Users/wilson/Downloads/artifacts%20(2)/offline.log
UTEC
Starting Offline at stage dev (us-east-1)
Offline [http for lambda] listening on http://localhost:3002
Function names exposed for local invocation by aws-sdk:
* api: gitlab-lambda-demo-dev-api
GET | http://localhost:3000/hello
POST | http://localhost:3000/2015-03-31/functions/api/invocations
GET | http://localhost:3000/add
POST | http://localhost:3000/2015-03-31/functions/api/invocations
Server ready: http://localhost:3000
GET /hello (λ: api)
(λ: api) RequestId: c43dc9d9-5b5d-486c-904d-5012b4e9e477 Duration: 310.72 ms Billed Duration: 311 ms
GET /hello (λ: api)
(λ: api) RequestId: 5b71a3cb-b228-4d15-abbf-110402aedb5c Duration: 1.63 ms Billed Duration: 2 ms
GET /add (λ: api)
(λ: api) RequestId: 68bbb94a-94bf-4a35-aded-8c956f880c51 Duration: 1.28 ms Billed Duration: 2 ms